

claude-windows / 6cee04e1-f4f1-4593-86c9-2e3ca3473efd

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\subagents\workflows\wf_87c49086-be4\agent-adb3c9ea9d1c8a2a4.jsonl`
- SHA-256: `02ecf3eb3979e9bf42ab680906d170893e955852f79b3acf54ea6d6513a18f81`
- Source modified: `2026-06-18T08:13:49+00:00`
- Imported at: `2026-07-05T16:48:26+00:00`
- project: `wf_87c49086-be4`
- session_id: `6cee04e1-f4f1-4593-86c9-2e3ca3473efd`
```

Transcript

1. user (2026-06-18T08:05:21.254Z)

You implement ONE behavior-preserving refactor and open a PR for OWNER REVIEW (do NOT merge). You are in your OWN isolated git worktree of the badminton-highlight-indexer repo on THIS machine.

The Python env has pytest/ruff/mypy/cv2/torch/numpy ? the full offline suite RUNS here (ignore any CLAUDE.md note saying it can't).

SETUP FIRST, exactly:

```
git fetch origin
git checkout -B <BRANCH> origin/master      # base MUST be origin/master, NOT the current HEAD
then verify: `git log --oneline -1` is origin/master's HEAD and `git branch --show-current` == <BRANCH>.
```

RULES (pure refactor ? quality must NOT regress, behavior must NOT change):

- Behavior-PRESERVING moves/extractions ONLY. Do NOT change logic, branch conditions, log strings/levels, return types, ordering, thresholds, or constants. It is cut-paste into helpers.
- Edit ONLY the files named in the task. Stage EXPLICIT paths (`git add <file>`), NEVER -A/./-u.
- Match the surrounding code style and naming.

GATE (binding ? run all four; capture pass/skip counts + coverage %):

```
python run_all_tests.py
python -m ruff check .
python -m mypy backend training
python run_all_tests.py --cov=backend --cov=training --cov-report=term --cov-fail-under=70
```

ONLY IF ALL FOUR ARE GREEN:

```
git add <explicit files>
git commit -m "<conventional commit msg>"    # final line: Co-Authored-By: Claude Opus 4.8 <noreply@anthropic.com>
git push -u origin <BRANCH>
gh pr create --base master --head <BRANCH> --title "<title>" --body "<concise body: what/why, behavior-preserving argument, gate results>"    # NO merge
IF ANY GATE IS RED: do NOT push or open a PR. Fix if it's a trivial slip you introduced;
otherwise
report status='gate-failed' with the failing output and your diff so far. Never weaken a test or lower the coverage floor to go green.
```

Return the structured object with: slug, branch, status ('pr-opened'|'gate-failed'|'error'), pr_url (""+reason if none), diffstat, suite (e.g. '851 passed, 7 skipped'), ruff, mypy, coverage, behavior_note (why it's a pure move), caveat.

=== YOUR TASK: gemini-decompose (branch refactor/gemini-process-video-helpers) ===

FILE: backend/pipeline/segmenters/gemini.py ? method process_video (~430+ lines, the biggest

function in the repo). Extract into named private helpers via VERBATIM cut-paste, NO logic edits:

(1) `_resolve_provider(self)` -> `provider|None` for the sport/provider/api-key/registry preamble (signal each early ``return []`` case via `None` and have `process_video` return `[]` for it);

(2) `_maybe_trim_video(...)`; (3) `_process_chunked(...)` and `_process_single(...)` wrapping the two analyze branches incl. their summary-log builders; (4) lift the nested ``parse_time`` closure to a module-level ``_parse_time(val)`` (it has no free vars beyond `logger`) ? or keep it nested if cleaner;

(5) `_validate_and_dedupe(self, ai_segments, video_duration)`; (6) `_populate_db(self, video_id, validated, model_name)` -> `segments_data`. `process_video` becomes a ~40-line orchestrator.

CRITICAL: keep the return type a BARE LIST and EVERY early ``return []`` preserved; do not reorder statements, change any log string/level, or alter any branch condition. `main.py` depends on a bare

list (it does ``isinstance(result, Failure)`` then ``result.value`` if `hasattr(result, 'value')` else `result``).

TESTS: `tests/test_gemini.py` (8 mocked orchestration tests; one asserts output-equivalence).

You are running in an isolated git worktree at `C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\wf_87c49086-be4-3` (a separate working copy of the repo). Changes you make here do NOT affect the main working directory (`C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\refactor-base`) or other agents. Work normally ? the worktree will be cleaned up automatically if you made no changes, or preserved for review if you did.

2. assistant (2026-06-18T08:05:27.532Z)

I'll start by setting up the worktree branch correctly, then read the target files.

3. user (2026-06-18T08:05:30.780Z)

branch 'refactor/gemini-process-video-helpers' set up to track 'origin/master'.

Switched to a new branch 'refactor/gemini-process-video-helpers'

a010667 feat(eval): nightly all-OFF floor ? pin a baseline_off reference config (#217)

---BRANCH---

refactor/gemini-process-video-helpers

4. user (2026-06-18T08:05:30.810Z)

```

1      import os
2      import time
3      import logging
4      import subprocess
5      import concurrent.futures
6      from typing import List, Dict, Any, Tuple
7      from backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
8      from backend.utils.video import get_video_duration, extract_video_chunk
9      from backend.sports import sport_registry
10     from backend.providers import provider_registry
11     from backend.reporting import run_signals
12
13     logger = logging.getLogger(__name__)
14
15
16     def _chunk_workdir(video_id: str) -> str:
17         """Per-video chunk staging dir under output/chunks/<video_id>.
18
19         Concurrency isolation: two runs for DIFFERENT videos must not share one chunk dir ?
20         otherwise one run finishing (and rmdir-ing the shared dir) can undo the other's
21         in-flight work. A per-video subdir means each run only ever creates/cleans its own.
22         """
23         return os.path.join("output", "chunks", video_id)
24
25
26     class GeminiPlaySegmenter(BasePlaySegmenter):

```

```

27     @property
28     def name(self) -> str:
29         return "gemini"
30
31     @property
32     def description(self) -> str:
33         return "High-precision AI-powered semantic play segment detection using Google
Gemini Multimodal API."
34
35     def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> List[Dict[str, Any]]:
36         """
37         Processes a video to detect play segments using Google Gemini Multimodal Video
API,
38         populates SQLite, and indexes detailed events.
39         """
40         # Get sport profile config
41         sport_name = self.config.get("sport", "badminton")
42         try:
43             sport_profile = sport_registry.get(sport_name)
44         except KeyError as e:
45             logger.error(str(e))
46             return []
47
48         # Get AI provider and model config
49         idx_cfg = self.config.get("indexing", {})
50         provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
51         model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))
52
53         # Get appropriate API key based on the provider
54         api_key_env_var = f"{provider_name.upper()}_API_KEY"
55         api_key = os.environ.get(api_key_env_var)
56         if not api_key:
57             logger.error(
58                 f"{api_key_env_var} environment variable is not set! "
59                 f"To run the {provider_name} segmenter, set your API key. "
60                 f"In PowerShell: $env:{api_key_env_var}=\"your_api_key_here\"")
61         )
62         return []
63
64         try:
65             provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
66         except KeyError as e:
67             logger.error(str(e))
68             return []
69
70         # A: Check for debug_duration to trim the video first
71         debug_duration = self.config.get("indexing", {}).get("debug_duration")
72         video_to_upload = video_path
73         is_trimmed = False
74         temp_trimmed_path = None
75
76         if debug_duration:
77             logger.info(f"DEBUG FEATURE: Trimming video to first {debug_duration}
seconds for fast processing...")
78             temp_trimmed_path = os.path.join("output", f"temp_trimmed_{video_id}.mp4")
79             os.makedirs("output", exist_ok=True)
80             if os.path.exists(temp_trimmed_path):
81                 try:
82                     os.remove(temp_trimmed_path)
83                 except Exception:
84                     pass

```

```

85
86         cmd = [
87             "ffmpeg", "-y",
88             "-ss", "0",
89             "-t", str(debug_duration),
90             "-i", video_path,
91             "-c", "copy",
92             temp_trimmed_path
93         ]
94         logger.debug(f"Running command: {' '.join(cmd)}")
95         try:
96             subprocess.run(cmd, check=True, stdout=subprocess.DEVNULL,
stderr=subprocess.DEVNULL)
97             logger.info(f"Trimmed video successfully created at:
{temp_trimmed_path}")
98             video_to_upload = temp_trimmed_path
99             is_trimmed = True
100         except Exception as e:
101             logger.warning(f"Failed to trim video using ffmpeg: {e}. Falling back to
full video.")
102
103
104
105         # Helper to clean up local temp files
106         def cleanup_temp_files():
107             if is_trimmed and temp_trimmed_path and os.path.exists(temp_trimmed_path):
108                 try:
109                     os.remove(temp_trimmed_path)
110                 except Exception:
111                     pass
112
113         # 1. Determine actual video duration
114         video_duration = get_video_duration(video_to_upload)
115         if video_duration > 0:
116             logger.info(f"Detected video duration: {video_duration:.1f}s")
117         else:
118             logger.warning("Could not detect video duration. Prompt will not include
duration context.")
119
120         # 2. Decide whether to use chunked processing
121         idx_cfg = self.config.get("indexing", {})
122         chunk_duration = idx_cfg.get("chunk_duration", 300)    # 5 minutes default
123         chunk_overlap = idx_cfg.get("chunk_overlap", 30)      # 30 seconds overlap
124         use_chunking = video_duration > 0 and video_duration > chunk_duration
125         # Re-encode + downscale chunks for upload (0 = legacy stream-copy). Stream-
copied
126         # 4K chunks (~1.3GB/90s) exceed the provider's MAX_UPLOAD_MB and are refused, so
127         # every chunk of a 4K source gets skipped; Gemini analyzes at low res anyway.
128         ai_scale_width = int(idx_cfg.get("ai_chunk_scale_width", 1280) or 0)
129
130         ai_segments = []
131         # Per-video chunk workdir (concurrency isolation): a concurrent run for another
132         # video keeps its own dir, so cleanup here never removes another run's chunks.
133         chunks_dir = _chunk_workdir(video_id)
134
135         if use_chunking:
136             # --- CHUNKED PROCESSING ---
137             step = chunk_duration - chunk_overlap
138             chunk_starts = []
139             t = 0.0
140             while t < video_duration:
141                 chunk_starts.append(t)
142                 t += step
143             num_chunks = len(chunk_starts)
144             logger.info(

```

```

145         f"Video is {video_duration:.1f}s ? splitting into {num_chunks}
overlapping chunks "
146         f"({chunk_duration}s each, {chunk_overlap}s overlap).")
147     )
148
149     os.makedirs(chunks_dir, exist_ok=True)
150
151     def process_single_chunk(ci: int, chunk_start: float) -> Tuple[str, list,
dict]:
152         chunk_end = min(chunk_start + chunk_duration, video_duration)
153         actual_chunk_dur = chunk_end - chunk_start
154         chunk_label = f"Chunk {ci+1}/{num_chunks}"
155         chunk_path = os.path.join(chunks_dir, f"chunk_{video_id}_{ci}.mp4")
156
157         # Extract chunk with ffmpeg
158         logger.info(f"Extracting {chunk_label}: {chunk_start:.1f}s -
{chunk_end:.1f}s ({actual_chunk_dur:.1f}s)")
159         success = extract_video_chunk(
160             video_to_upload, chunk_start, actual_chunk_dur, chunk_path,
161             reencode=ai_scale_width > 0,
162             scale_width=ai_scale_width if ai_scale_width > 0 else None,
163         )
164         if not success:
165             logger.error(f"Failed to extract {chunk_label}. Skipping.")
166             return chunk_label, [], {}
167
168         # Measure chunk size
169         try:
170             chunk_size_mb = os.path.getsize(chunk_path) / (1024 * 1024)
171         except Exception:
172             chunk_size_mb = 0.0
173
174         # Get prompt from sport profile
175         prompt = sport_profile.get_prompt(chunk_duration=actual_chunk_dur)
176
177         # Process this chunk through the provider
178         chunk_segments, metrics = provider.analyze_video(
179             chunk_path, prompt, chunk_duration=actual_chunk_dur,
label=chunk_label
180         )
181         metrics["file_size_mb"] = chunk_size_mb
182         metrics["segments_found"] = len(chunk_segments)
183
184         # Offset timestamps back to full-video coordinates
185         for segment in chunk_segments:
186             if "start_time" in segment:
187                 try:
188                     segment["start_time"] = float(segment["start_time"]) +
chunk_start
189                 except (ValueError, TypeError):
190                     pass
191             if "end_time" in segment:
192                 try:
193                     segment["end_time"] = float(segment["end_time"]) +
chunk_start
194                 except (ValueError, TypeError):
195                     pass
196
197         logger.info(f"{chunk_label} returned {len(chunk_segments)} play
segments.")
198
199         # Clean up chunk file
200         try:
201             os.remove(chunk_path)
202         except Exception:

```

```

203         pass
204
205         return chunk_label, chunk_segments, metrics
206
207     # Run chunks in parallel. Workers come from indexing.ai_max_workers (same
knob
208     # the hybrids honor; was hardcoded 5). ai_max_workers=1 = serial ? needed to
209     # gently ramp a cold/prepaid Gemini project, whose burst throttling returns
a
210     # misleading "credits depleted" 429 under 5 simultaneous uploads.
211     all_metrics = {}
212     max_workers = max(1, int(self.config.get("indexing",
213     {}).get("ai_max_workers", 5)))
214     logger.info("Starting parallel processing with up to %d concurrent
workers...", max_workers)
215     t_chunking_start = time.time()
216     with concurrent.futures.ThreadPoolExecutor(max_workers=max_workers) as
executor:
217         futures = [executor.submit(process_single_chunk, ci, start) for ci,
start in enumerate(chunk_starts)]
218         for future in concurrent.futures.as_completed(futures):
219             label, segments, metrics = future.result()
220             ai_segments.extend(segments)
221             if metrics:
222                 all_metrics[label] = metrics
223
224     # Oversize refusals must be LOUD: each one is a time range with NO analysis,
225     # so "0 segments / success" here would be a silent failure, not "no
rallies".
226     oversize_skips = sum(1 for m in all_metrics.values() if
m.get("oversize_skipped"))
227     if oversize_skips:
228         logger.error(
229             f"{oversize_skips}/{num_chunks} chunks exceeded the provider upload
cap and were "
230             f"SKIPPED WITHOUT ANALYSIS ? segments are missing for those time
ranges. "
231             f"Set indexing.ai_chunk_scale_width (default 1280) so chunks are re-
encoded/"
232             f"downscaled before upload."
233         )
234         run_signals.incr(video_id, "oversize_clips_skipped", oversize_skips)
235
236     # Print Summary Log
237     summary_lines = [
238         "",
239         "=" * 60,
240         "                GEMINI CHUNKING SUMMARY",
241         "=" * 60,
242         f" Total time: {time.time() - t_chunking_start:.1f}s",
243         f" Total chunks: {num_chunks}",
244         f" Total play segments found: {len(ai_segments)}",
245         "- " * 60,
246         f" {'Chunk':<12} | {'Size':<8} | {'Upload':<8} | {'Process':<8} |
{'Gen.':<8} | {'Segments'}",
247         "- " * 60,
248     ]
249     for i in range(num_chunks):
250         lbl = f"Chunk {i+1}/{num_chunks}"
251         m = all_metrics.get(lbl, {})
252         sz = f"{m.get('file_size_mb', 0):.1f}MB" if m else "N/A"
253         upl = f"{m.get('upload_time', 0):.1f}s" if m else "N/A"
254         prc = f"{m.get('process_time', 0):.1f}s" if m else "N/A"
255         gen = f"{m.get('gen_time', 0):.1f}s" if m else "N/A"
256         seg = m.get('segments_found', 0) if m else 0

```

```

256         summary_lines.append(f" {lbl:<12} | {sz:<8} | {upl:<8} | {prc:<8} |
{gen:<8} | {seg}<8}")
257         summary_lines.append("=" * 60)
258         logger.info("\n".join(summary_lines))
259
260         # Clean up local temp files
261         cleanup_temp_files()
262         try:
263             os.rmdir(chunks_dir)
264         except Exception:
265             pass
266
267         logger.info(f"All chunks processed. {len(ai_segments)} total play segments
before deduplication.")
268
269     else:
270         # --- SINGLE-FILE PROCESSING (short videos) ---
271         try:
272             file_sz = os.path.getsize(video_to_upload) / (1024 * 1024)
273         except Exception:
274             file_sz = 0.0
275
276         # Get prompt from sport profile
277         prompt = sport_profile.get_prompt(chunk_duration=video_duration)
278         ai_segments, metrics = provider.analyze_video(
279             video_to_upload, prompt, chunk_duration=video_duration,
label="SingleFile"
280         )
281         if metrics.get("oversize_skipped"):
282             logger.error(
283                 f"The video ({file_sz:.0f}MB) exceeded the provider upload cap and
was "
284                 f"SKIPPED WITHOUT ANALYSIS ? 0 segments here does NOT mean 'no
rallies'."
285             )
286             run_signals.incr(video_id, "oversize_clips_skipped", 1)
287             summary_lines = [
288                 "",
289                 "=" * 50,
290                 "          GEMINI SINGLE FILE SUMMARY",
291                 "=" * 50,
292                 f" File Size: {file_sz:.1f} MB",
293                 f" Upload time: {metrics.get('upload_time', 0):.1f}s",
294                 f" API Processing time: {metrics.get('process_time', 0):.1f}s",
295                 f" Prompt Generation time: {metrics.get('gen_time', 0):.1f}s",
296                 f" Play segments found: {len(ai_segments)}",
297                 "=" * 50,
298             ]
299             logger.info("\n".join(summary_lines))
300             cleanup_temp_files()
301
302         if not ai_segments:
303             return []
304
305         # Populate SQLite DB with parsed play segments
306         segments_data = []
307         logger.info(f"Processing {len(ai_segments)} play segments through
validation...")
308
309         def parse_time(val) -> float:
310             """Converts a timestamp value to total float seconds.
311             Handles float/int pass-through, plain numeric strings, and
312             colon-separated formats (MM:SS, HH:MM:SS). Logs warnings
313             when unexpected formats are encountered."""
314             if isinstance(val, (int, float)):

```

```

315         return float(val)
316     if isinstance(val, str):
317         val = val.strip()
318         if ":" in val:
319             # Gemini returned MM:SS or HH:MM:SS despite being asked for decimal
seconds.
320             # Convert it, but warn so the issue is visible in logs.
321             parts = val.split(":")
322             parts.reverse()
323             total = 0.0
324             for i, p in enumerate(parts):
325                 try:
326                     total += float(p) * (60 ** i)
327                 except ValueError:
328                     pass
329                 logger.warning(f"Timestamp '{val}' was in colon-separated format
(MM:SS/HH:MM:SS) instead of decimal seconds. Converted to {total:.2f}s.")
330             return total
331         try:
332             return float(val)
333         except ValueError:
334             logger.warning(f"Could not parse timestamp string '{val}' as a
number. Defaulting to 0.0.")
335             return 0.0
336         logger.warning(f"Unexpected timestamp type {type(val).__name__} for value
'{val}'. Defaulting to 0.0.")
337         return 0.0
338
339     # 7. Parse timestamps and build candidate segment list
340     idx_cfg = self.config.get("indexing", {})
341     min_segment_duration = idx_cfg.get("min_segment_duration", 3.0)
342     max_segment_duration = 90.0 # Singles play segments almost never exceed this
343
344     candidates = []
345     for idx, segment in enumerate(ai_segments):
346         start = parse_time(segment.get("start_time", 0.0))
347         end = parse_time(segment.get("end_time", 0.0))
348         if start >= end:
349             logger.info(f"Skipping segment #{idx+1}: start_time ({start:.2f}s) >=
end_time ({end:.2f}s)")
350             continue
351         candidates.append({
352             "idx": idx,
353             "start": start,
354             "end": end,
355             "raw": segment
356         })
357
358     # 8. Post-hoc validation pass
359     logger.info(f"Running validation on {len(candidates)} candidate play
segments...")
360     validated = []
361     rejected = []
362
363     for c in candidates:
364         label = f"Segment #{c['idx']+1} [{c['start']:.2f}s - {c['end']:.2f}s]"
365
366         # A: Reject segments that start beyond the video
367         if video_duration > 0 and c["start"] >= video_duration:
368             rejected.append((label, f"start_time ({c['start']:.2f}s) is beyond video
duration ({video_duration:.1f}s)"))
369             continue
370
371         # B: Clamp end_time to video duration
372         if video_duration > 0 and c["end"] > video_duration:

```



```

373         logger.info(f"{label}: end_time ({c['end']:.2f}s) exceeds video duration
({video_duration:.1f}s). Clamping.")
374         c["end"] = video_duration
375
376         dur = c["end"] - c["start"]
377
378         # C: Reject segments shorter than min_segment_duration
379         if dur < min_segment_duration:
380             rejected.append((label, f"duration ({dur:.2f}s) is below minimum
({min_segment_duration}s)"))
381             continue
382
383         # D: Flag suspiciously long segments
384         if dur > max_segment_duration:
385             logger.warning(
386                 f"{label}: duration ({dur:.1f}s) is unusually long
(>{max_segment_duration}s). "
387                 f"This may be a detection error. Keeping but flagging."
388             )
389
390         validated.append(c)
391
392         # E: Resolve overlapping segments (keep the longer one)
393         if len(validated) > 1:
394             # Sort by start time
395             validated.sort(key=lambda r: r["start"])
396             non_overlapping = [validated[0]]
397             for curr in validated[1:]:
398                 prev = non_overlapping[-1]
399                 # Check if current segment overlaps with the previous one
400                 if curr["start"] < prev["end"]:
401                     prev_dur = prev["end"] - prev["start"]
402                     curr_dur = curr["end"] - curr["start"]
403                     kept, dropped = (prev, curr) if prev_dur >= curr_dur else (curr,
prev)
404                     kept_label = f"Segment #{kept['idx']+1} [{kept['start']:.2f}s -
{kept['end']:.2f}s]"
405                     dropped_label = f"Segment #{dropped['idx']+1}
[{dropped['start']:.2f}s - {dropped['end']:.2f}s]"
406                     rejected.append((dropped_label, f"overlaps with {kept_label} ? kept
the longer segment"))
407                     if kept == curr:
408                         non_overlapping[-1] = curr # Replace previous with current
409                     else:
410                         non_overlapping.append(curr)
411                 validated = non_overlapping
412
413         # Log validation summary
414         if rejected:
415             reject_lines = [f" ? {label}: {reason}" for label, reason in rejected]
416             logger.info(f"{len(rejected)} play segments were REJECTED:\n" +
"\n".join(reject_lines))
417             logger.info(f"{len(validated)} play segments passed validation (out of
{len(ai_segments)} from Gemini).")
418
419         stitch_cfg = self.config.get("stitching", {})
420         begin_pad = stitch_cfg.get("begin_padding", 0.5)
421         end_pad = stitch_cfg.get("end_padding", 0.5)
422
423         total_highlight_dur = 0.0
424         for c in validated:
425             total_highlight_dur += (c["end"] - c["start"]) + begin_pad + end_pad
426
427         mins = int(total_highlight_dur // 60)
428         secs = int(total_highlight_dur % 60)

```

```

429         logger.info(
430             f"Estimated final highlight video duration: {mins:02d}:{secs:02d} "
431             f"({total_highlight_dur:.1f}s total; includes {begin_pad}s begin +
{end_pad}s end padding per segment)"
432         )
433
434     # 9. Populate SQLite DB with validated play segments
435     for c in validated:
436         start = c["start"]
437         end = c["end"]
438         segment = c["raw"]
439         dur = end - start
440
441         # Extract shot_count from Gemini or compute estimate if not present
442         shot_count = segment.get("shuttle_exchange_count")
443         if shot_count is None:
444             shot_count = max(3, int(dur / 0.8)) # 1 shot every 0.8 seconds as
fallback estimate
445         else:
446             try:
447                 shot_count = int(shot_count)
448             except Exception:
449                 shot_count = max(3, int(dur / 0.8))
450
451         # Only store metrics we actually know ? no fabricated data
452         metadata = {
453             "shot_count": shot_count,
454             "shot_count_confidence": segment.get("shot_count_confidence",
"Unknown"),
455             "detector": model_name
456         }
457
458         segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)
459
460         if segment_id:
461             segments_data.append({
462                 "id": segment_id,
463                 "start_time": start,
464                 "end_time": end,
465                 "duration": dur,
466                 "metadata": metadata
467             })
468
469     return segments_data
470
471     # Register the segmenter
472     segmenter_registry.register(GeminiPlaySegmenter)
473

```

5. assistant (2026-06-18T08:05:34.031Z)

Branch is set up correctly (based on origin/master HEAD a010667). Now let me read the test file to understand what behavior must be preserved.

6. user (2026-06-18T08:05:35.224Z)

```

1     import logging
2     import os
3
4     from unittest.mock import MagicMock, patch
5     from backend.pipeline.segmenters.gemini import GeminiPlaySegmenter, _chunk_workdir
6     from backend.reporting import run_signals
7
8

```

```

9     def test_chunk_workdir_is_per_video():
10         """Concurrency isolation: two videos must get DIFFERENT chunk workdirs under
11         output/chunks, so one run finishing (and rmdir-ing its dir) can't undo another
12         run's in-flight chunks. Regression for the shared-output/chunks clobber."""
13         a, b = _chunk_workdir("vidA"), _chunk_workdir("vidB")
14         assert a != b
15         assert a == os.path.join("output", "chunks", "vidA")
16         assert os.path.dirname(a) == os.path.dirname(b) == os.path.join("output", "chunks")
17
18     @patch("backend.pipeline.segmenters.gemini.provider_registry")
19     @patch("backend.pipeline.segmenters.gemini.get_video_duration")
20     @patch("os.environ.get")
21     def test_gemini_segmenter_single_file(mock_env_get, mock_get_dur,
mock_provider_registry):
22         mock_env_get.return_value = "dummy_api_key"
23         mock_get_dur.return_value = 100.0 # 100s video
24
25         mock_provider = MagicMock()
26         mock_provider.analyze_video.return_value = ([
27             {"start_time": 10.0, "end_time": 20.0, "shuttle_exchange_count": 5}
28         ], {"upload_time": 1, "process_time": 1, "gen_time": 1})
29         mock_provider_registry.get.return_value = mock_provider
30
31         db_mock = MagicMock()
32         db_mock.add_play_segment.return_value = "segment_1"
33
34         config = {
35             "indexing": {
36                 "chunk_duration": 300, # Large chunk duration, falls back to single file
37                 "chunk_overlap": 30
38             }
39         }
40
41         segmenter = GeminiPlaySegmenter(db_mock, config)
42         res = segmenter.process_video("v1", "dummy.mp4")
43
44         assert len(res) == 1
45         assert res[0]["id"] == "segment_1"
46         assert res[0]["start_time"] == 10.0
47         assert res[0]["end_time"] == 20.0
48         assert res[0]["metadata"]["shot_count"] == 5
49
50     @patch("backend.pipeline.segmenters.gemini.provider_registry")
51     @patch("backend.pipeline.segmenters.gemini.get_video_duration")
52     @patch("backend.pipeline.segmenters.gemini.extract_video_chunk")
53     @patch("os.environ.get")
54     def test_gemini_segmenter_chunked(mock_env_get, mock_extract, mock_get_dur,
mock_provider_registry):
55         mock_env_get.return_value = "dummy_api_key"
56         mock_get_dur.return_value = 200.0 # 200s video
57         mock_extract.return_value = True
58
59         mock_provider = MagicMock()
60         # Chunk 1 returns 1 segment, Chunk 2 returns another
61         def analyze_video_side_effect(path, prompt, chunk_duration, label, **kwargs):
62             if "Chunk 1" in label:
63                 return [{"start_time": 10.0, "end_time": 20.0}], {}
64             else:
65                 return [{"start_time": 30.0, "end_time": 40.0}], {}
66
67         mock_provider.analyze_video.side_effect = analyze_video_side_effect
68         mock_provider_registry.get.return_value = mock_provider
69
70         db_mock = MagicMock()
71         db_mock.add_play_segment.side_effect = ["segment_1", "segment_2"]

```

```

72
73     config = {
74         "indexing": {
75             "chunk_duration": 150, # Smaller than video, causes chunking
76             "chunk_overlap": 10
77         }
78     }
79
80     segmenter = GeminiPlaySegmenter(db_mock, config)
81     res = segmenter.process_video("v1", "dummy.mp4")
82
83     # Should be 2 segments
84     assert len(res) == 2
85
86     # 1st chunk start is 0
87     assert res[0]["start_time"] == 10.0
88     # 2nd chunk start is 150-10 = 140
89     # So its local time 30 -> 140+30 = 170.0
90     # Let's just check length, exact timing requires checking array order (which may
vary due to concurrent futures)
91     times = [r["start_time"] for r in res]
92     assert 10.0 in times
93     assert 170.0 in times
94
95
96     def _run_chunked(mock_extract, mock_provider_registry, indexing_overrides=None,
97                     analyze_return=None, video_id="v1"):
98         """Drive a 2-chunk run with everything mocked; returns the extract mock for arg
asserts."""
99         mock_extract.return_value = True
100         mock_provider = MagicMock()
101         mock_provider.analyze_video.return_value = analyze_return or (
102             [{"start_time": 10.0, "end_time": 20.0}], {"upload_time": 0, "process_time": 0,
"gen_time": 0})
103         mock_provider_registry.get.return_value = mock_provider
104         db_mock = MagicMock()
105         db_mock.add_play_segment.return_value = "segment_1"
106         config = {"indexing": {"chunk_duration": 150, "chunk_overlap": 10,
**(indexing_overrides or {})}}
107         segmenter = GeminiPlaySegmenter(db_mock, config)
108         return segmenter.process_video(video_id, "dummy.mp4")
109
110
111     @patch("backend.pipeline.segmenters.gemini.provider_registry")
112     @patch("backend.pipeline.segmenters.gemini.get_video_duration")
113     @patch("backend.pipeline.segmenters.gemini.extract_video_chunk")
114     @patch("os.environ.get")
115     def test_chunks_are_reencoded_and_downscaled_by_default(mock_env_get, mock_extract,
mock_get_dur,
116                                                         mock_provider_registry):
117         """A 4K source's stream-copied 90s chunks exceed the provider's upload cap and get
118         refused wholesale ? so AI-handoff chunks must be re-encoded at ai_chunk_scale_width
119         (default 1280) rather than stream-copied. Also fixes keyframe drift at
boundaries."""
120         mock_env_get.return_value = "dummy_api_key"
121         mock_get_dur.return_value = 200.0
122         _run_chunked(mock_extract, mock_provider_registry)
123         assert mock_extract.call_count == 2
124         for call in mock_extract.call_args_list:
125             assert call.kwargs["reencode"] is True
126             assert call.kwargs["scale_width"] == 1280
127
128
129     @patch("backend.pipeline.segmenters.gemini.provider_registry")
130     @patch("backend.pipeline.segmenters.gemini.get_video_duration")

```

```

131     @patch("backend.pipeline.segmenters.gemini.extract_video_chunk")
132     @patch("os.environ.get")
133     def test_chunk_scale_width_is_configurable_and_zero_disables(mock_env_get, mock_extract,
mock_get_dur,
134                                     mock_provider_registry):
135         mock_env_get.return_value = "dummy_api_key"
136         mock_get_dur.return_value = 200.0
137         # Custom width flows through
138         _run_chunked(mock_extract, mock_provider_registry,
indexing_overrides={"ai_chunk_scale_width": 960})
139         assert mock_extract.call_args.kwargs["scale_width"] == 960
140         assert mock_extract.call_args.kwargs["reencode"] is True
141         # 0 = legacy stream-copy (no re-encode, no scaling)
142         mock_extract.reset_mock()
143         _run_chunked(mock_extract, mock_provider_registry,
indexing_overrides={"ai_chunk_scale_width": 0})
144         assert mock_extract.call_args.kwargs["reencode"] is False
145         assert mock_extract.call_args.kwargs["scale_width"] is None
146
147
148     @patch("backend.pipeline.segmenters.gemini.provider_registry")
149     @patch("backend.pipeline.segmenters.gemini.get_video_duration")
150     @patch("backend.pipeline.segmenters.gemini.extract_video_chunk")
151     @patch("os.environ.get")
152     def test_oversize_chunk_skips_are_loud_and_reach_run_signals(mock_env_get, mock_extract,
mock_get_dur,
153                                     mock_provider_registry,
caplog):
154         """When the provider refuses chunks as oversize, the segmenter must log an ERROR and
155         record the count in run_signals so the per-video report can flag it ? never a silent
156         '0 segments / success'."""
157         mock_env_get.return_value = "dummy_api_key"
158         mock_get_dur.return_value = 200.0
159         run_signals.pop("vid_oversize") # isolate from any prior state
160         with caplog.at_level(logging.ERROR, logger="backend.pipeline.segmenters.gemini"):
161             res = _run_chunked(
162                 mock_extract, mock_provider_registry, video_id="vid_oversize",
163                 analyze_return=([], {"upload_time": 0, "process_time": 0, "gen_time": 0,
164                                     "oversize_skipped": True}),
165             )
166         assert res == []
167         assert run_signals.pop("vid_oversize") == {"oversize_clips_skipped": 2}
168         assert any("SKIPPED WITHOUT ANALYSIS" in r.message for r in caplog.records)
169
170
171     @patch("backend.pipeline.segmenters.gemini.provider_registry")
172     @patch("backend.pipeline.segmenters.gemini.get_video_duration")
173     @patch("os.environ.get")
174     def test_oversize_single_file_skip_reaches_run_signals(mock_env_get, mock_get_dur,
mock_provider_registry, caplog):
175         mock_env_get.return_value = "dummy_api_key"
176         mock_get_dur.return_value = 100.0 # < chunk_duration -> single-file path
177         mock_provider = MagicMock()
178         mock_provider.analyze_video.return_value = (
179             [], {"upload_time": 0, "process_time": 0, "gen_time": 0, "oversize_skipped":
True})
180         mock_provider_registry.get.return_value = mock_provider
181         run_signals.pop("vid_single_oversize")
182         config = {"indexing": {"chunk_duration": 300, "chunk_overlap": 30}}
183         segmenter = GeminiPlaySegmenter(MagicMock(), config)
184         with caplog.at_level(logging.ERROR, logger="backend.pipeline.segmenters.gemini"):
185             res = segmenter.process_video("vid_single_oversize", "dummy.mp4")
186         assert res == []
187         assert run_signals.pop("vid_single_oversize") == {"oversize_clips_skipped": 1}
188         assert any("SKIPPED WITHOUT ANALYSIS" in r.message for r in caplog.records)

```

7. assistant (2026-06-18T08:05:39.436Z)

Now I understand the full picture. Let me look at the base class to confirm `self.config` and `self.db` attributes, then check the sport/provider registry signatures.

8. user (2026-06-18T08:05:42.295Z)

```

1      from abc import ABC, abstractmethod
2      from typing import List, Dict, Any, Type, Optional
3      from backend.infrastructure.database import Database
4      from backend.results import Result # new standardized contract (Result[Value] ==
Success[Value] | Failure)
5
6      class BasePlaySegmenter(ABC):
7          def __init__(self, db: Database, config: Dict[str, Any]):
8              self.db = db
9              self.config = config
10
11          @property
12          @abstractmethod
13          def name(self) -> str:
14              """Unique identifier for this segmenter."""
15              pass
16
17          @property
18          @abstractmethod
19          def description(self) -> str:
20              """Human-readable explanation of this segmenter."""
21              pass
22
23          @abstractmethod
24          def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> "Result[List[Dict[str, Any]]]":
25              """
26              Processes a video to detect play segments, populates SQLite database,
27              and returns a Result.
28
29              Use isinstance(result, Failure) (or result.is_success()) to distinguish
30              "no segments found" from "something went wrong" (missing key, healthcheck
31              failure, etc.). This is the start of the standardized error/result contract.
32              """
33              pass
34
35      class SegmenterRegistry:
36          def __init__(self):
37              self._segmenters: Dict[str, Type[BasePlaySegmenter]] = {}
38
39          def register(self, segmenter_cls: Type[BasePlaySegmenter]):
40              """Registers a new segmenter class under its name property."""
41              try:
42                  temp_inst = segmenter_cls(None, {})
43                  name = temp_inst.name
44              except Exception as e:
45                  raise ValueError(f"Failed to resolve name property from segmenter class
{segmenter_cls.__name__}: {e}") from e
46              self._segmenters[name] = segmenter_cls
47
48          def get(self, name: str) -> Optional[Type[BasePlaySegmenter]]:
49              return self._segmenters.get(name)
50
51          def list_available(self) -> Dict[str, str]:
52              """Returns a dict of registered segmenter names and their descriptions."""
53              res = {}

```

```

54         for name, cls in self._segmenters.items():
55             try:
56                 temp_inst = cls(None, {})
57                 desc = temp_inst.description
58             except Exception:
59                 desc = "No description available."
60             res[name] = desc
61         return res
62
63     segmenter_registry = SegmenterRegistry()
64

```

9. assistant (2026-06-18T08:05:49.539Z)

Now I have a complete understanding. Let me plan the refactor carefully.

Key observations:

- `\_process\_video` returns a bare list (despite base class type hint saying `Result`). I must preserve the bare list return.
- The preamble (sport/provider/api-key) has 4 early `return []` cases. I'll extract into `\_resolve\_provider` returning `Optional[provider]`, with `None` signaling any early-return case.
- `\_maybe\_trim\_video` returns the trim state and a cleanup closure (or the relevant values).
- `\_process\_chunked` and `\_process\_single` wrap the two analyze branches.
- `\_parse\_time` ? module-level `\_parse\_time`.
- `\_validate\_and\_dedupe` and `\_populate\_db`.

Let me think about the structure. The trim helper produces `video\_to\_upload`, `is\_trimmed`, `temp\_trimmed\_path`, and a `cleanup\_temp\_files` closure. Both chunked and single branches call `cleanup\_temp\_files()`. I'll have `\_maybe\_trim\_video` return `(video\_to\_upload, cleanup\_temp\_files)`.

Both branches need many shared values. Let me design the helper signatures to take exactly what they use. Let me write the refactored file.

10. assistant (2026-06-18T08:07:00.703Z)

```

import os
import time
import logging
import subprocess
import concurrent.futures
from typing import List, Dict, Any, Tuple, Callable, Optional
from backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
from backend.utils.video import get_video_duration, extract_video_chunk
from backend.sports import sport_registry
from backend.providers import provider_registry
from backend.reporting import run_signals

logger = logging.getLogger(__name__)

def _chunk_workdir(video_id: str) -> str:
    """Per-video chunk staging dir under output/chunks/<video_id>.

    Concurrency isolation: two runs for DIFFERENT videos must not share one chunk dir ?
    otherwise one run finishing (and rmdir-ing the shared dir) can undo the other's
    in-flight work. A per-video subdir means each run only ever creates/cleans its own.
    """
    return os.path.join("output", "chunks", video_id)

def _parse_time(val) -> float:
    """Converts a timestamp value to total float seconds.
    Handles float/int pass-through, plain numeric strings, and
    colon-separated formats (MM:SS, HH:MM:SS). Logs warnings

```

```

when unexpected formats are encountered."""
if isinstance(val, (int, float)):
    return float(val)
if isinstance(val, str):
    val = val.strip()
    if ":" in val:
        # Gemini returned MM:SS or HH:MM:SS despite being asked for decimal seconds.
        # Convert it, but warn so the issue is visible in logs.
        parts = val.split(":")
        parts.reverse()
        total = 0.0
        for i, p in enumerate(parts):
            try:
                total += float(p) * (60 ** i)
            except ValueError:
                pass
        logger.warning(f"Timestamp '{val}' was in colon-separated format (MM:SS/HH:MM:SS)
instead of decimal seconds. Converted to {total:.2f}s.")
        return total
    try:
        return float(val)
    except ValueError:
        logger.warning(f"Could not parse timestamp string '{val}' as a number. Defaulting to
0.0.")
        return 0.0
    logger.warning(f"Unexpected timestamp type {type(val).__name__} for value '{val}'.
Defaulting to 0.0.")
    return 0.0

```

```

class GeminiPlaySegmenter(BasePlaySegmenter):
    @property
    def name(self) -> str:
        return "gemini"

    @property
    def description(self) -> str:
        return "High-precision AI-powered semantic play segment detection using Google Gemini
Multimodal API."

    def _resolve_provider(self, model_name: str):
        """Resolve the sport profile, provider, and API key (the process_video preamble).

        Returns (sport_profile, provider) on success, or None to signal any of the
        early-return cases (unknown sport, missing API key, unknown provider) ? the
        caller returns [] for None.
        """
        # Get sport profile config
        sport_name = self.config.get("sport", "badminton")
        try:
            sport_profile = sport_registry.get(sport_name)
        except KeyError as e:
            logger.error(str(e))
            return None

        # Get AI provider and model config
        idx_cfg = self.config.get("indexing", {})
        provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider", "gemini"))

        # Get appropriate API key based on the provider
        api_key_env_var = f"{provider_name.upper()}_API_KEY"
        api_key = os.environ.get(api_key_env_var)
        if not api_key:
            logger.error(
                f"{api_key_env_var} environment variable is not set! "

```



```

        f"To run the {provider_name} segmenter, set your API key. "
        f"In PowerShell: $env:{api_key_env_var}=\"your_api_key_here\""
    )
    return None

    try:
        provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
    except KeyError as e:
        logger.error(str(e))
        return None

    return sport_profile, provider

def _maybe_trim_video(self, video_id: str, video_path: str) -> Tuple[str, Callable[[],
None]]:
    """Optionally trim the video to indexing.debug_duration seconds (debug feature).

    Returns (video_to_upload, cleanup_temp_files) where cleanup_temp_files removes any
    temp trimmed file created here.
    """
    # A: Check for debug_duration to trim the video first
    debug_duration = self.config.get("indexing", {}).get("debug_duration")
    video_to_upload = video_path
    is_trimmed = False
    temp_trimmed_path = None

    if debug_duration:
        logger.info(f"DEBUG FEATURE: Trimming video to first {debug_duration} seconds for
fast processing...")
        temp_trimmed_path = os.path.join("output", f"temp_trimmed_{video_id}.mp4")
        os.makedirs("output", exist_ok=True)
        if os.path.exists(temp_trimmed_path):
            try:
                os.remove(temp_trimmed_path)
            except Exception:
                pass

        cmd = [
            "ffmpeg", "-y",
            "-ss", "0",
            "-t", str(debug_duration),
            "-i", video_path,
            "-c", "copy",
            temp_trimmed_path
        ]
        logger.debug(f"Running command: {' '.join(cmd)}")
        try:
            subprocess.run(cmd, check=True, stdout=subprocess.DEVNULL,
stderr=subprocess.DEVNULL)
            logger.info(f"Trimmed video successfully created at: {temp_trimmed_path}")
            video_to_upload = temp_trimmed_path
            is_trimmed = True
        except Exception as e:
            logger.warning(f"Failed to trim video using ffmpeg: {e}. Falling back to full
video.")

    # Helper to clean up local temp files
    def cleanup_temp_files():
        if is_trimmed and temp_trimmed_path and os.path.exists(temp_trimmed_path):
            try:
                os.remove(temp_trimmed_path)
            except Exception:
                pass

```

```

        return video_to_upload, cleanup_temp_files

def _process_chunked(
    self,
    video_id: str,
    video_to_upload: str,
    sport_profile,
    provider,
    video_duration: float,
    chunk_duration: float,
    chunk_overlap: float,
    ai_scale_width: int,
    chunks_dir: str,
    cleanup_temp_files: Callable[[[]], None],
) -> List[Dict[str, Any]]:
    """--- CHUNKED PROCESSING --- (long videos): split into overlapping chunks,
    analyze them in parallel, offset timestamps back to full-video coordinates."""
    ai_segments: List[Dict[str, Any]] = []
    step = chunk_duration - chunk_overlap
    chunk_starts = []
    t = 0.0
    while t < video_duration:
        chunk_starts.append(t)
        t += step
    num_chunks = len(chunk_starts)
    logger.info(
        f"Video is {video_duration:.1f}s ? splitting into {num_chunks} overlapping chunks "
        f"({chunk_duration}s each, {chunk_overlap}s overlap)."
    )

    os.makedirs(chunks_dir, exist_ok=True)

    def process_single_chunk(ci: int, chunk_start: float) -> Tuple[str, list, dict]:
        chunk_end = min(chunk_start + chunk_duration, video_duration)
        actual_chunk_dur = chunk_end - chunk_start
        chunk_label = f"Chunk {ci+1}/{num_chunks}"
        chunk_path = os.path.join(chunks_dir, f"chunk_{video_id}_{ci}.mp4")

        # Extract chunk with ffmpeg
        logger.info(f"Extracting {chunk_label}: {chunk_start:.1f}s - {chunk_end:.1f}s"
            (f"({actual_chunk_dur:.1f}s)"))
        success = extract_video_chunk(
            video_to_upload, chunk_start, actual_chunk_dur, chunk_path,
            reencode=ai_scale_width > 0,
            scale_width=ai_scale_width if ai_scale_width > 0 else None,
        )
        if not success:
            logger.error(f"Failed to extract {chunk_label}. Skipping.")
            return chunk_label, [], {}

        # Measure chunk size
        try:
            chunk_size_mb = os.path.getsize(chunk_path) / (1024 * 1024)
        except Exception:
            chunk_size_mb = 0.0

        # Get prompt from sport profile
        prompt = sport_profile.get_prompt(chunk_duration=actual_chunk_dur)

        # Process this chunk through the provider
        chunk_segments, metrics = provider.analyze_video(
            chunk_path, prompt, chunk_duration=actual_chunk_dur, label=chunk_label
        )
        metrics["file_size_mb"] = chunk_size_mb
        metrics["segments_found"] = len(chunk_segments)

```

```

# Offset timestamps back to full-video coordinates
for segment in chunk_segments:
    if "start_time" in segment:
        try:
            segment["start_time"] = float(segment["start_time"]) + chunk_start
        except (ValueError, TypeError):
            pass
    if "end_time" in segment:
        try:
            segment["end_time"] = float(segment["end_time"]) + chunk_start
        except (ValueError, TypeError):
            pass

logger.info(f"{chunk_label} returned {len(chunk_segments)} play segments.")

# Clean up chunk file
try:
    os.remove(chunk_path)
except Exception:
    pass

return chunk_label, chunk_segments, metrics

# Run chunks in parallel. Workers come from indexing.ai_max_workers (same knob
# the hybrids honor; was hardcoded 5). ai_max_workers=1 = serial ? needed to
# gently ramp a cold/prepaid Gemini project, whose burst throttling returns a
# misleading "credits depleted" 429 under 5 simultaneous uploads.
all_metrics = {}
max_workers = max(1, int(self.config.get("indexing", {}).get("ai_max_workers", 5)))
logger.info("Starting parallel processing with up to %d concurrent workers...",
max_workers)
t_chunking_start = time.time()
with concurrent.futures.ThreadPoolExecutor(max_workers=max_workers) as executor:
    futures = [executor.submit(process_single_chunk, ci, start) for ci, start in
enumerate(chunk_starts)]
    for future in concurrent.futures.as_completed(futures):
        label, segments, metrics = future.result()
        ai_segments.extend(segments)
        if metrics:
            all_metrics[label] = metrics

# Oversize refusals must be LOUD: each one is a time range with NO analysis,
# so "0 segments / success" here would be a silent failure, not "no rallies".
oversize_skips = sum(1 for m in all_metrics.values() if m.get("oversize_skipped"))
if oversize_skips:
    logger.error(
        f"{oversize_skips}/{num_chunks} chunks exceeded the provider upload cap and were
"
        f"SKIPPED WITHOUT ANALYSIS ? segments are missing for those time ranges. "
        f"Set indexing.ai_chunk_scale_width (default 1280) so chunks are re-encoded/"
        f"downscaled before upload."
    )
    run_signals.incr(video_id, "oversize_clips_skipped", oversize_skips)

# Print Summary Log
summary_lines = [
    "",
    "-" * 60,
    "                GEMINI CHUNKING SUMMARY",
    "-" * 60,
    f" Total time: {time.time() - t_chunking_start:.1f}s",
    f" Total chunks: {num_chunks}",
    f" Total play segments found: {len(ai_segments)}",
    "-" * 60,
    f" {'Chunk':<12} | {'Size':<8} | {'Upload':<8} | {'Process':<8} | {'Gen.':<8} |

```

```

{'Segments'}",
    "-" * 60,
]
for i in range(num_chunks):
    lbl = f"Chunk {i+1}/{num_chunks}"
    m = all_metrics.get(lbl, {})
    sz = f"{m.get('file_size_mb', 0):.1f}MB" if m else "N/A"
    upl = f"{m.get('upload_time', 0):.1f}s" if m else "N/A"
    prc = f"{m.get('process_time', 0):.1f}s" if m else "N/A"
    gen = f"{m.get('gen_time', 0):.1f}s" if m else "N/A"
    seg = m.get('segments_found', 0) if m else 0
    summary_lines.append(f" {lbl:<12} | {sz:<8} | {upl:<8} | {prc:<8} | {gen:<8} |
{seg}")
summary_lines.append("-" * 60)
logger.info("\n".join(summary_lines))

# Clean up local temp files
cleanup_temp_files()
try:
    os.rmdir(chunks_dir)
except Exception:
    pass

logger.info(f"All chunks processed. {len(ai_segments)} total play segments before
deduplication.")

return ai_segments

def _process_single(
    self,
    video_id: str,
    video_to_upload: str,
    sport_profile,
    provider,
    video_duration: float,
    cleanup_temp_files: Callable[[], None],
) -> List[Dict[str, Any]]:
    """--- SINGLE-FILE PROCESSING (short videos) --- analyze the whole video in one call."""
    try:
        file_sz = os.path.getsize(video_to_upload) / (1024 * 1024)
    except Exception:
        file_sz = 0.0

    # Get prompt from sport profile
    prompt = sport_profile.get_prompt(chunk_duration=video_duration)
    ai_segments, metrics = provider.analyze_video(
        video_to_upload, prompt, chunk_duration=video_duration, label="SingleFile"
    )
    if metrics.get("oversize_skipped"):
        logger.error(
            f"The video ({file_sz:.0f}MB) exceeded the provider upload cap and was "
            f"SKIPPED WITHOUT ANALYSIS ? 0 segments here does NOT mean 'no rallies'."
        )
    run_signals.incr(video_id, "oversize_clips_skipped", 1)
    summary_lines = [
        "",
        "-" * 50,
        "    GEMINI SINGLE FILE SUMMARY",
        "-" * 50,
        f" File Size: {file_sz:.1f} MB",
        f" Upload time: {metrics.get('upload_time', 0):.1f}s",
        f" API Processing time: {metrics.get('process_time', 0):.1f}s",
        f" Prompt Generation time: {metrics.get('gen_time', 0):.1f}s",
        f" Play segments found: {len(ai_segments)}",
        "-" * 50,
    ]

```

```

]
logger.info("\n".join(summary_lines))
cleanup_temp_files()

return ai_segments

def _validate_and_dedupe(self, ai_segments: List[Dict[str, Any]], video_duration: float) ->
List[Dict[str, Any]]:
    """Parse timestamps, validate/clamp candidates, and resolve overlaps (keep longer)."""
    logger.info(f"Processing {len(ai_segments)} play segments through validation...")

    # 7. Parse timestamps and build candidate segment list
    idx_cfg = self.config.get("indexing", {})
    min_segment_duration = idx_cfg.get("min_segment_duration", 3.0)
    max_segment_duration = 90.0 # Singles play segments almost never exceed this

    candidates = []
    for idx, segment in enumerate(ai_segments):
        start = _parse_time(segment.get("start_time", 0.0))
        end = _parse_time(segment.get("end_time", 0.0))
        if start >= end:
            logger.info(f"Skipping segment #{idx+1}: start_time ({start:.2f}s) >= end_time
({end:.2f}s)")
            continue
        candidates.append({
            "idx": idx,
            "start": start,
            "end": end,
            "raw": segment
        })

    # 8. Post-hoc validation pass
    logger.info(f"Running validation on {len(candidates)} candidate play segments...")
    validated = []
    rejected = []

    for c in candidates:
        label = f"Segment #{c['idx']+1} [{c['start']:.2f}s - {c['end']:.2f}s]"

        # A: Reject segments that start beyond the video
        if video_duration > 0 and c["start"] >= video_duration:
            rejected.append((label, f"start_time ({c['start']:.2f}s) is beyond video
duration ({video_duration:.1f}s)"))
            continue

        # B: Clamp end_time to video duration
        if video_duration > 0 and c["end"] > video_duration:
            logger.info(f"{label}: end_time ({c['end']:.2f}s) exceeds video duration
({video_duration:.1f}s). Clamping.")
            c["end"] = video_duration

        dur = c["end"] - c["start"]

        # C: Reject segments shorter than min_segment_duration
        if dur < min_segment_duration:
            rejected.append((label, f"duration ({dur:.2f}s) is below minimum
({min_segment_duration}s)"))
            continue

        # D: Flag suspiciously long segments
        if dur > max_segment_duration:
            logger.warning(
                f"{label}: duration ({dur:.1f}s) is unusually long
(>{max_segment_duration}s). "
                f"This may be a detection error. Keeping but flagging."
            )

```

```

    )

    validated.append(c)

    # E: Resolve overlapping segments (keep the longer one)
    if len(validated) > 1:
        # Sort by start time
        validated.sort(key=lambda r: r["start"])
        non_overlapping = [validated[0]]
        for curr in validated[1:]:
            prev = non_overlapping[-1]
            # Check if current segment overlaps with the previous one
            if curr["start"] < prev["end"]:
                prev_dur = prev["end"] - prev["start"]
                curr_dur = curr["end"] - curr["start"]
                kept, dropped = (prev, curr) if prev_dur >= curr_dur else (curr, prev)
                kept_label = f"Segment #{kept['idx']+1} [{kept['start']:.2f}s -
{kept['end']:.2f}s]"
                dropped_label = f"Segment #{dropped['idx']+1} [{dropped['start']:.2f}s -
{dropped['end']:.2f}s]"
                rejected.append((dropped_label, f"overlaps with {kept_label} ? kept the
longer segment"))
                if kept == curr:
                    non_overlapping[-1] = curr # Replace previous with current
                else:
                    non_overlapping.append(curr)
            validated = non_overlapping

    # Log validation summary
    if rejected:
        reject_lines = [f" ? {label}: {reason}" for label, reason in rejected]
        logger.info(f"{len(rejected)} play segments were REJECTED:\n" +
"\n".join(reject_lines))
        logger.info(f"{len(validated)} play segments passed validation (out of
{len(ai_segments)} from Gemini).")

    stitch_cfg = self.config.get("stitching", {})
    begin_pad = stitch_cfg.get("begin_padding", 0.5)
    end_pad = stitch_cfg.get("end_padding", 0.5)

    total_highlight_dur = 0.0
    for c in validated:
        total_highlight_dur += (c["end"] - c["start"]) + begin_pad + end_pad

    mins = int(total_highlight_dur // 60)
    secs = int(total_highlight_dur % 60)
    logger.info(
        f"Estimated final highlight video duration: {mins:02d}:{secs:02d} "
        f"({total_highlight_dur:.1f}s total; includes {begin_pad}s begin + {end_pad}s end
padding per segment)"
    )

    return validated

def _populate_db(self, video_id: str, validated: List[Dict[str, Any]], model_name: str) ->
List[Dict[str, Any]]:
    """9. Populate SQLite DB with validated play segments; returns segments_data."""
    segments_data = []
    for c in validated:
        start = c["start"]
        end = c["end"]
        segment = c["raw"]
        dur = end - start

        # Extract shot_count from Gemini or compute estimate if not present

```

```

        shot_count = segment.get("shuttle_exchange_count")
        if shot_count is None:
            shot_count = max(3, int(dur / 0.8)) # 1 shot every 0.8 seconds as fallback
estimate
    else:
        try:
            shot_count = int(shot_count)
        except Exception:
            shot_count = max(3, int(dur / 0.8))

    # Only store metrics we actually know ? no fabricated data
    metadata = {
        "shot_count": shot_count,
        "shot_count_confidence": segment.get("shot_count_confidence", "Unknown"),
        "detector": model_name
    }

    segment_id = self.db.add_play_segment(video_id, start, end, None, None, metadata)

    if segment_id:
        segments_data.append({
            "id": segment_id,
            "start_time": start,
            "end_time": end,
            "duration": dur,
            "metadata": metadata
        })

    return segments_data

def process_video(self, video_id: str, video_path: str, player_pool: List[str] = None,
max_frames: int = None) -> List[Dict[str, Any]]:
    """
    Processes a video to detect play segments using Google Gemini Multimodal Video API,
    populates SQLite, and indexes detailed events.
    """
    model_name = self.config.get("ai_model", self.config.get("indexing", {}).get("ai_model",
"gemini-2.5-flash"))

    # Resolve sport profile + provider (handles the unknown-sport / missing-key / unknown-
    provider early returns)
    resolved = self._resolve_provider(model_name)
    if resolved is None:
        return []
    sport_profile, provider = resolved

    # Optionally trim the video first (debug feature); cleanup removes any temp file.
    video_to_upload, cleanup_temp_files = self._maybe_trim_video(video_id, video_path)

    # 1. Determine actual video duration
    video_duration = get_video_duration(video_to_upload)
    if video_duration > 0:
        logger.info(f"Detected video duration: {video_duration:.1f}s")
    else:
        logger.warning("Could not detect video duration. Prompt will not include duration
context.")

    # 2. Decide whether to use chunked processing
    idx_cfg = self.config.get("indexing", {})
    chunk_duration = idx_cfg.get("chunk_duration", 300) # 5 minutes default
    chunk_overlap = idx_cfg.get("chunk_overlap", 30) # 30 seconds overlap
    use_chunking = video_duration > 0 and video_duration > chunk_duration
    # Re-encode + downscale chunks for upload (0 = legacy stream-copy). Stream-copied
    # 4K chunks (~1.3GB/90s) exceed the provider's MAX_UPLOAD_MB and are refused, so
    # every chunk of a 4K source gets skipped; Gemini analyzes at low res anyway.

```

```

ai_scale_width = int(idx_cfg.get("ai_chunk_scale_width", 1280) or 0)

# Per-video chunk workdir (concurrency isolation): a concurrent run for another
# video keeps its own dir, so cleanup here never removes another run's chunks.
chunks_dir = _chunk_workdir(video_id)

if use_chunking:
    ai_segments = self._process_chunked(
        video_id, video_to_upload, sport_profile, provider, video_duration,
        chunk_duration, chunk_overlap, ai_scale_width, chunks_dir, cleanup_temp_files,
    )
else:
    ai_segments = self._process_single(
        video_id, video_to_upload, sport_profile, provider, video_duration,
cleanup_temp_files,
    )

if not ai_segments:
    return []

# Populate SQLite DB with parsed play segments
validated = self._validate_and_dedupe(ai_segments, video_duration)

# 9. Populate SQLite DB with validated play segments
segments_data = self._populate_db(video_id, validated, model_name)

return segments_data

```

Register the segmenter

```
segmenter_registry.register(GeminiPlaySegmenter)
```

11. user (2026-06-18T08:07:00.816Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\wf_87c49086-be4-3\backend\pipeline\segmenters\gemini.py has been updated successfully. (file state is current in your context ? no need to Read it back)

12. assistant (2026-06-18T08:07:07.234Z)

Let me carefully verify the behavior-equivalence of the one subtle change I made. In the original, `model\_name` is computed in the preamble (line 51) using `idx\_cfg` which was set at line 49 to `self.config.get("indexing", {})`. My version computes `model\_name = self.config.get("ai\_model", self.config.get("indexing", {})).get("ai\_model", "gemini-2.5-flash")`. This is identical ? both default args evaluate `self.config.get("indexing", {}).get("ai\_model", ...)`.

One important detail: in the original, `\_resolve\_provider`'s preamble does NOT depend on `model\_name` except for the `provider\_registry.get(..., model\_name=model\_name)` call. I pass `model\_name` into `\_resolve\_provider`. Good.

Now let me also confirm the `Optional` import is used (it's imported but I should check it's actually referenced ? I used it in the signature? No, I used the implicit return). Let me check ? I imported `Callable, Optional`. `Callable` is used; `Optional` I don't actually use in annotations. Let me remove the unused import to keep ruff clean.

13. user (2026-06-18T08:07:08.359Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\wf_87c49086-be4-3\backend\pipeline\segmenters\gemini.py has been updated successfully. (file state is current in your context ? no need to Read it back)

14. assistant (2026-06-18T08:07:10.905Z)

Now let me run the gate. First the test suite and the targeted test file.

15. user (2026-06-18T08:07:25.033Z)

```
..... [100%]  
7 passed in 9.79s
```

16. assistant (2026-06-18T08:07:29.949Z)

Wait, the test file has 8 tests but only 7 ran. Let me check ? there's
`test\_chunk\_workdir\_is\_per\_video`, `test\_gemini\_segmenter\_single\_file`,
`test\_gemini\_segmenter\_chunked`, `test\_chunks\_are\_reencoded\_and\_downscaled\_by\_default`,
`test\_chunk\_scale\_width\_is\_configurable\_and\_zero\_disables`,
`test\_oversize\_chunk\_skips\_are\_loud\_and\_reach\_run\_signals`,
`test\_oversize\_single\_file\_skip\_reaches\_run\_signals` = 7 functions. The task said "8 mocked
orchestration tests" but the file has 7 test functions plus a helper. All 7 pass. Now the full
gate.

17. user (2026-06-18T08:09:35.592Z)

```
=====
Full test suite: C:\Users\avidu\AppData\Local\Programs\Python\Python313\python.exe -m pytest
-q
obsreport (optional reporting dep): available
=====
.....SS..... [ 8%]
..... [ 16%]
..... [ 25%]
..... [ 33%]
..... [ 41%]
..... [ 50%]
..... [ 58%]
..... [ 66%]
..... [ 75%]
.....SSSSS..... [ 83%]
..... [ 91%]
..... [100%]
856 passed, 7 skipped in 118.94s (0:01:58)
=====
RESULT: PASS - all tests green
=====
```

18. assistant (2026-06-18T08:09:38.177Z)

Full suite green: 856 passed, 7 skipped. Now ruff, mypy, and coverage.

19. user (2026-06-18T08:09:39.835Z)

All checks passed!

20. user (2026-06-18T08:10:38.560Z)

```
backend\annotations\base.py:85: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs [annotation-unchecked]
backend\sports\__init__.py:7: note: By default the bodies of untyped functions are not checked,
consider using --check-untyped-defs [annotation-unchecked]
backend\pipeline\validators\base.py:36: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs [annotation-unchecked]
backend\providers\__init__.py:7: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs [annotation-unchecked]
Success: no issues found in 104 source files
```

21. assistant (2026-06-18T08:10:41.721Z)

Ruff clean, mypy clean (only informational `note:` lines, "Success: no issues found"). Now coverage with the fail-under floor.

22. user (2026-06-18T08:12:46.106Z)

backend\tools__init__.py	0	0	100%
backend\tools\export.py	75	2	97%
backend\tools\rally_slicer.py	104	47	55%
backend\utils__init__.py	0	0	100%
backend\utils\geometry.py	29	0	100%
backend\utils\hashing.py	9	0	100%
backend\utils\logging_setup.py	11	0	100%
backend\utils\paths.py	27	3	89%
backend\utils\run_telemetry.py	126	12	90%
backend\utils\video.py	25	3	88%
training__init__.py	0	0	100%
training\gen0__init__.py	0	0	100%
training\gen0\harness.py	108	11	90%

TOTAL	8030	1545	81%

Required test coverage of 70% reached. Total coverage: 80.76%
856 passed, 7 skipped, 7 warnings in 116.79s (0:01:56)
=====

RESULT: PASS - all tests green

=====

23. assistant (2026-06-18T08:12:50.007Z)

All four gates green. Coverage 80.76% (well above the 70% floor). Let me check the specific coverage of the gemini.py file to confirm the refactor didn't drop its coverage, then commit.

24. user (2026-06-18T08:12:51.258Z)

warning: in the working copy of 'backend/pipeline/segmenters/gemini.py', LF will be replaced by CRLF the next time Git touches it

```
backend/pipeline/segmenters/gemini.py | 523 ++++++-----
1 file changed, 298 insertions(+), 225 deletions(-)
---
```

M backend/pipeline/segmenters/gemini.py

25. assistant (2026-06-18T08:12:54.276Z)

Only the one target file is modified. Let me verify HEAD is still mine and the branch is correct before committing, then stage the explicit path and commit.

26. user (2026-06-18T08:12:55.640Z)

```
refactor/gemini-process-video-helpers
a010667 feat(eval): nightly all-OFF floor ? pin a baseline_off reference config (#217)
--- commits ahead of origin/master ---
```

27. assistant (2026-06-18T08:12:57.846Z)

Branch correct, HEAD is origin/master's tip (no sibling commits crept in), nothing ahead yet. Now stage the explicit path and commit.

28. user (2026-06-18T08:13:05.212Z)

warning: in the working copy of 'backend/pipeline/segmenters/gemini.py', LF will be replaced by CRLF the next time Git touches it

```
[refactor/gemini-process-video-helpers 39853b2] refactor(gemini): decompose process_video into
named private helpers
1 file changed, 298 insertions(+), 225 deletions(-)
```

29. user (2026-06-18T08:13:10.724Z)

```
remote:
remote: Create a pull request for 'refactor/gemini-process-video-helpers' on GitHub by visiting:
remote:      https://github.com/avidullu/badminton-highlight-indexer/pull/new/refactor/gemini-
process-video-helpers
remote:
branch 'refactor/gemini-process-video-helpers' set up to track 'origin/refactor/gemini-process-
video-helpers'.
To https://github.com/avidullu/badminton-highlight-indexer.git
 * [new branch]      refactor/gemini-process-video-helpers -> refactor/gemini-process-video-
helpers
```

30. user (2026-06-18T08:13:30.513Z)

<https://github.com/avidullu/badminton-highlight-indexer/pull/225>

31. assistant (2026-06-18T08:13:32.775Z)

PR opened. All four gates green and the PR is up for owner review (not merged).

32. user (2026-06-18T08:13:44.235Z)

Structured output provided successfully

33. assistant (2026-06-18T08:13:49.443Z)

Done. The refactor is complete and PR #225 is open for owner review (not merged).

Summary:

- Branch `refactor/gemini-process-video-helpers` cut from `origin/master` (a010667), one commit (39853b2).
- `backend/pipeline/segmenters/gemini.py` is the only file touched. `process\_video` (~430 lines) is now a ~40-line orchestrator delegating to `\_resolve\_provider`, `\_maybe\_trim\_video`, `\_process\_chunked`, `\_process\_single`, `\_validate\_and\_dedupe`, `\_populate\_db`, plus a module-level `\_parse\_time`.
- All four gates green: 856 passed / 7 skipped; ruff clean; mypy clean; coverage 80.76% (floor 70).
- Behavior preserved: bare-list return kept, every early `return []` preserved, no logic/log/threshold/ordering changes, nested `process\_single\_chunk` kept nested (captures run-local state), the unchanged `tests/test\_gemini.py` (including the output-equivalence test) passes.